

1. Introduction

In today's digital era, automation plays a crucial role in reducing manual effort and increasing efficiency in business operations. The jewellery and bullion market is highly dynamic, where gold and silver prices fluctuate frequently throughout the day. Shop owners are required to update these rates to their customers regularly, especially through messaging platforms like WhatsApp. However, manual updating is time-consuming, error-prone, and inefficient.

To address this problem, the Bullion Auto-Pilot System is developed. This system is a web-based automation tool designed to fetch live gold and silver rates from online sources, apply custom margins as per shop requirements, and automatically send the updated rates to a specified WhatsApp group at scheduled times.

The system is built using Python technologies such as Flask for the web interface, Selenium for web scraping, and PyWhatKit for WhatsApp automation. It allows users to configure parameters like group ID, gold and silver margins, and update timings through a simple graphical interface.

The application runs in the background and performs scheduled tasks such as:

- Fetching real-time bullion rates from trusted websites
- Adjusting prices using user-defined margins
- Sending formatted messages to WhatsApp groups automatically

This system eliminates the need for repetitive manual work, ensures timely updates, and improves communication efficiency between shop owners and customers.

2. Problem Definition

In the bullion and jewellery business, gold and silver prices change frequently throughout the day. Shop owners need to regularly update these rates to their customers, usually through messaging platforms such as WhatsApp. This process is traditionally performed manually, which leads to several challenges.

Firstly, manually checking live rates from different websites consumes time and requires continuous monitoring. Secondly, shop owners often apply their own margins to the base rates, which increases the chances of calculation errors. Thirdly, sending updates repeatedly to WhatsApp groups at different times of the day becomes a repetitive and tedious task.

Due to these issues, there is a high possibility of:

- Delayed updates to customers
- Incorrect rate calculations
- Increased workload for the shop owner
- Inefficient communication process

Therefore, there is a need for an automated system that can fetch real-time bullion rates, apply custom margins accurately, and send updates to WhatsApp groups at scheduled times without manual intervention.

The proposed system aims to eliminate these problems by providing a reliable and automated solution for rate management and communication.

3. User Interview

Q1: What is the most frustrating part of sending daily rate updates manually?

- *Answer:* Remembering to do it exactly on time. When the shop is crowded with customers, I completely forget to check the rates and message the WhatsApp group, which delays business.

Q2: How do you currently calculate the rates before sending them to customers?

- *Answer:* I look at the live market rate on a website, open my calculator, add my shop's making charge/margin, and then type it out on WhatsApp. Doing this 3-4 times a day is tiring and sometimes leads to calculation typos.

Q3: Do you want a complex web system with cloud logins and databases?

- *Answer:* No. I just want a simple, standalone software on my shop's billing laptop. I should just be able to open it, put in today's margin, set the time, and let it run in the background.

Q4: What websites do you trust for pulling live market rates?

- *Answer:* Safari Bullions for Gold and Shree Navratna for Silver. The bot must pull data exactly from these trusted sources.

Q5: How do you feel about the security of using an automated bot with your business WhatsApp account?

- *Answer:* Security is very important. I do not want to hand over my WhatsApp data to third-party cloud APIs. If the bot runs locally on my own PC using my own Chrome browser, I feel completely safe.

Q6: What happens if the internet is slow or the rate website doesn't load immediately?

- *Answer:* The bot shouldn't just crash. It should wait a few seconds and try again so that the daily update isn't missed.

4. Feasibility Study

Feasibility study is used to determine whether the proposed system is practical and beneficial for the organization. It evaluates different factors such as user acceptance, cost, environment, and technical requirements before implementing the system.

For the WhatsApp Automation System, the feasibility study includes the following aspects.

4.1 User Feasibility

The software is designed with a "Dark Mode" modern GUI. The user simply types in the margins, sets the schedule, and clicks "Start Bot". It requires zero coding knowledge, making it highly feasible for non-technical shop owners.

4.1.1 Environmental Feasibility

The system is environment-friendly as it reduces manual work and paper usage. All operations such as rate checking and message sending are performed digitally. It does not cause any harm to the environment and promotes efficient digital communication.

4.1.2 Economical Feasibility

The system is cost-effective as it uses open-source technologies like Python, Flask, and Selenium. There is no need for expensive hardware or software. It reduces manpower effort and saves time, which indirectly reduces operational costs for shop owners.

4.1.3 Technical Feasibility

The system is technically feasible as it is built using well-known and reliable technologies such as Python, Selenium, and web-based automation tools. The required tools and libraries are easily available and can run on a standard computer system. The implementation of scheduling and automation is achievable without complex infrastructure.

4.1.3.1 Hardware Feasibility:-

The system does not require any special hardware. It can run on a normal computer or laptop with basic specifications such as:

- Minimum 4GB RAM
- Internet connection
- Web browser

Hence, it is easy to deploy and use.

4.1.3.2 Software Feasibility

The software requirements for the system include:

- Python programming language
- Flask framework
- Selenium WebDriver
- Web browser (Google Chrome)

All these software tools are freely available and easy to install, making the system highly feasible from a software perspective.

5. System Requirements Specification

System Requirement Specification (SRS) describes the overall working of the system, including the existing system, its limitations, and the proposed system with improved features.

5.1: Current Working

In the current system, bullion rate updates are performed manually by shop owners. The process involves visiting different websites to check live gold and silver rates, calculating the final price by adding margins, and sending updates to customers via WhatsApp.

This system has several limitations:

- Time-consuming process of checking rates multiple times a day
- Manual calculation of margins, which may lead to errors
- Repetitive task of sending messages to WhatsApp groups
- Possibility of delayed updates due to busy schedules
- No automation or scheduling support

Overall, the current system is inefficient and heavily dependent on manual effort.

5.2: Proposed Working

The proposed system is an automated web-based application that simplifies the entire process of bullion rate updates.

In this system:

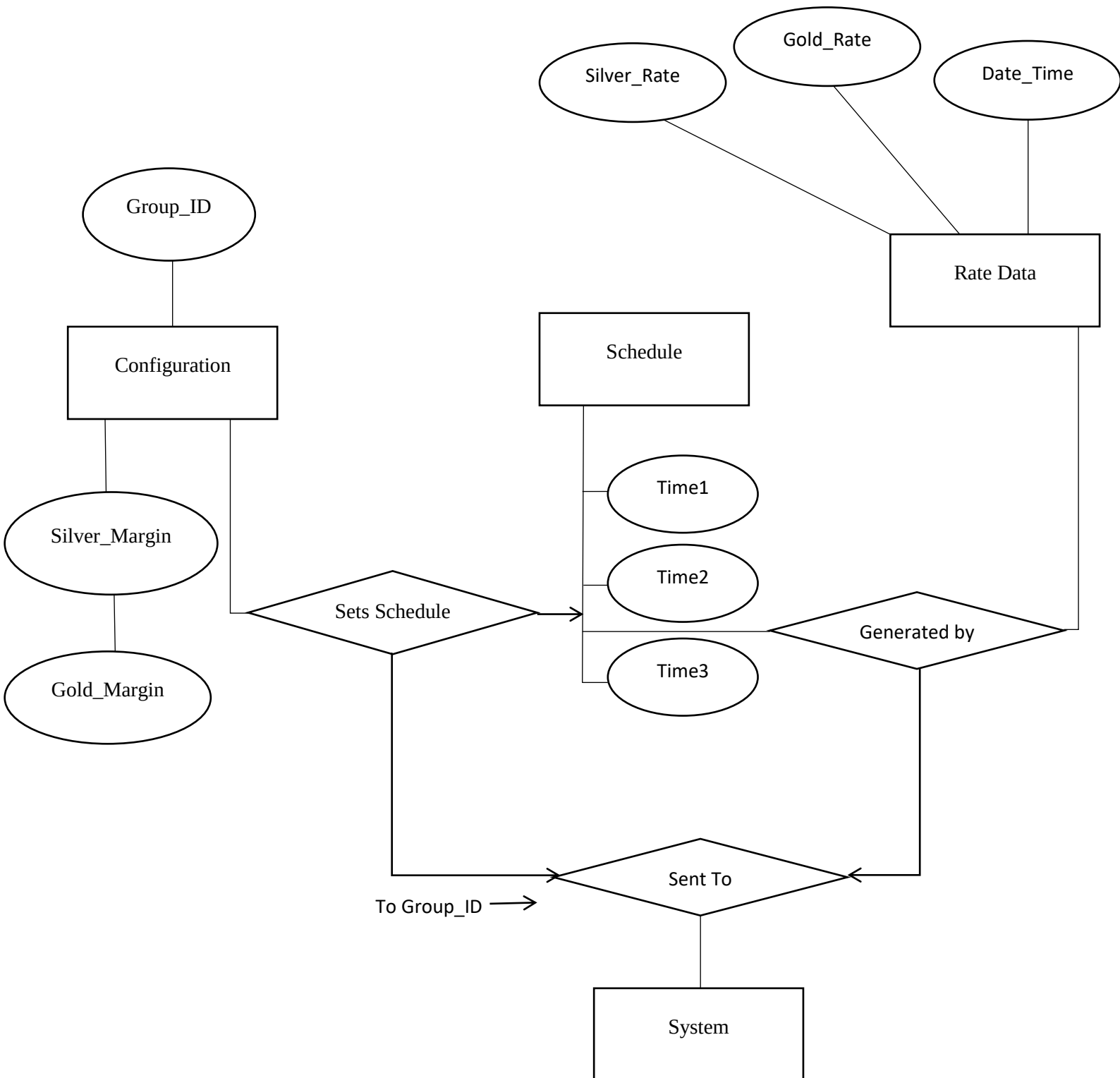
- Live gold and silver rates are automatically fetched from online sources using web scraping
- User-defined margins are added automatically to the fetched rates
- The system sends formatted messages to a specified WhatsApp group

- Updates are scheduled at predefined times (morning, afternoon, evening)
- A web interface is provided to configure settings such as group ID, margins, and timings

This system reduces manual work, improves accuracy, and ensures timely communication with customers.

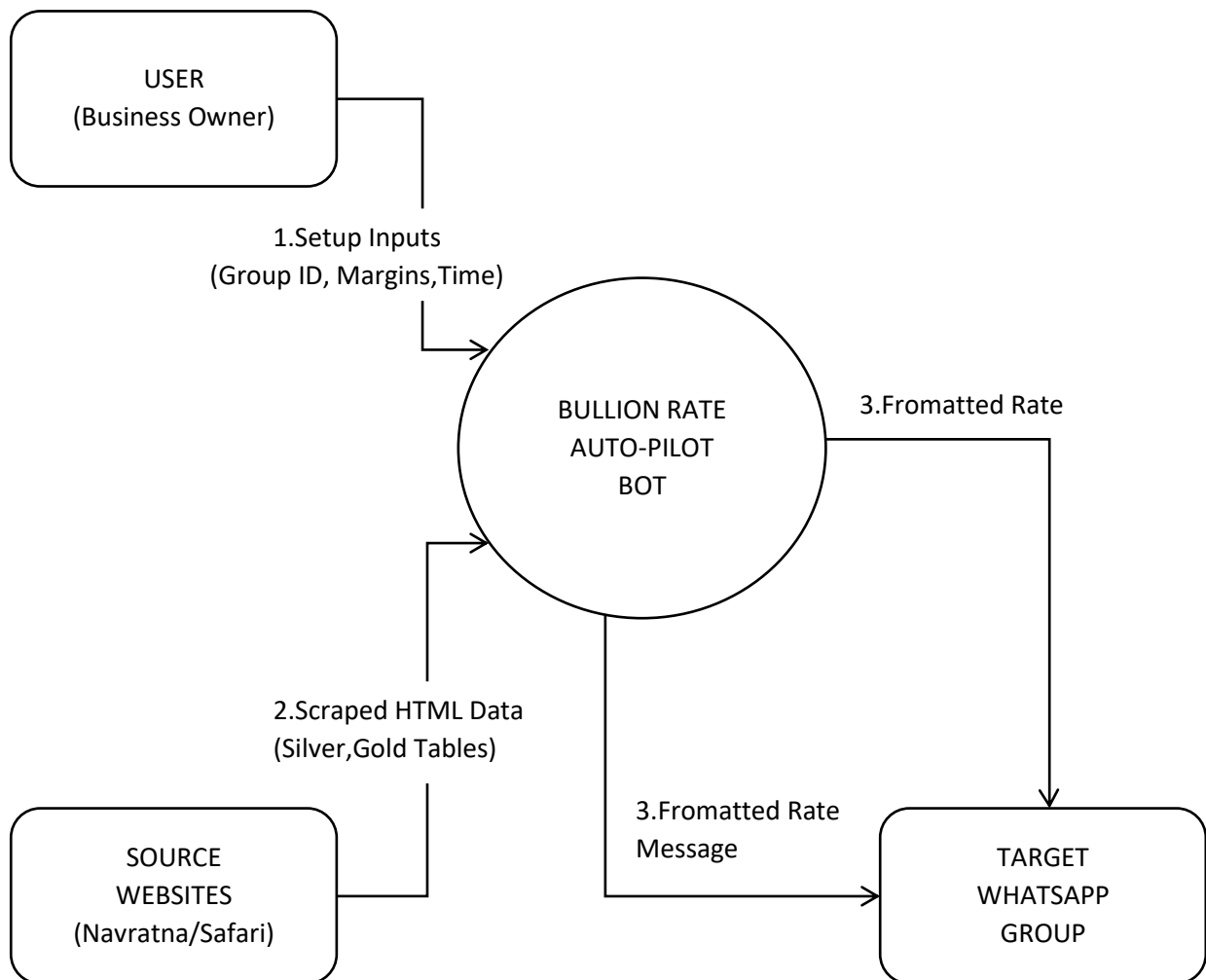
6. ER Diagram

-the system does not use a Database

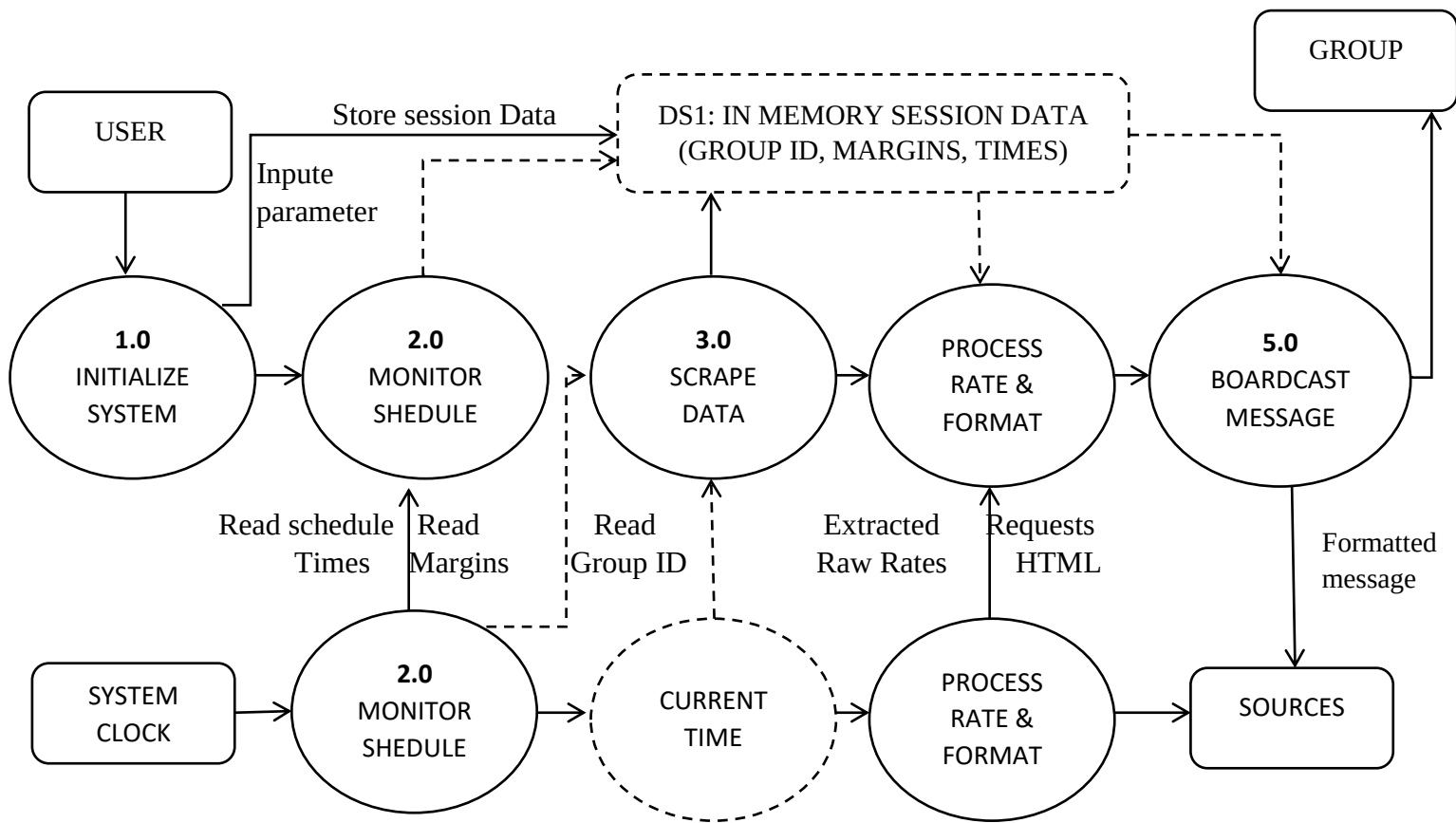


7. Data flow diagram

DFD: level 0



DFD: level 1 : Process Decomposition



8. Table Structure (internal Data State)

Although the current system works without a database and uses in-memory data processing, a logical table structure is designed for future scalability and data storage.

1. Table Name : Configuration Table

This table stores user inputs such as group ID and margin values.

Field Name	Data Type	Description
Config_ID	INT(PK)	Unique identifier
Group_ID	VARCHAR	WhatsApp Group ID
Silver_Margin	INT	Margin added to silver rate
Gold_Margin	INT	Margin added to gold rate

2. Table Name: Schedule Table

This table stores the time schedule for sending updates.

Field Name	Data Type	Description
Schedule_ID	INT(PK)	Unique identifier
Time1	TIME	Morning update time
Time2	TIME	Afternoon update time
Time3	TIME	Evening update time

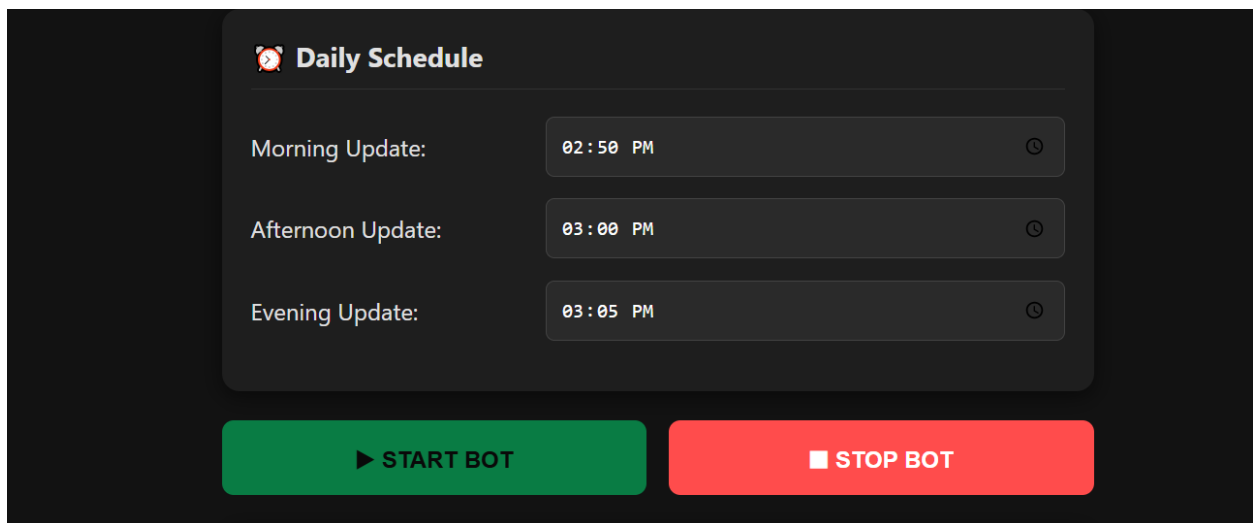
3. Table Name: Rate data Table

This table stores both raw and processed rates.

Field Name	Data Type	Description
Rate_ID	INT(PK)	Unique identifier
Raw_Silver_Rate	INT	Scraped silver rate
Raw_Gold_Rate	INT	Scraped gold rate
Final_Silver_Rate	INT	Silver rate after margin
Final_Gold_Rate	INT	Gold rate after margin
Date_Time	DATETIME	Timestamp of rate generation

9. Screens

1.Schedule start stop button

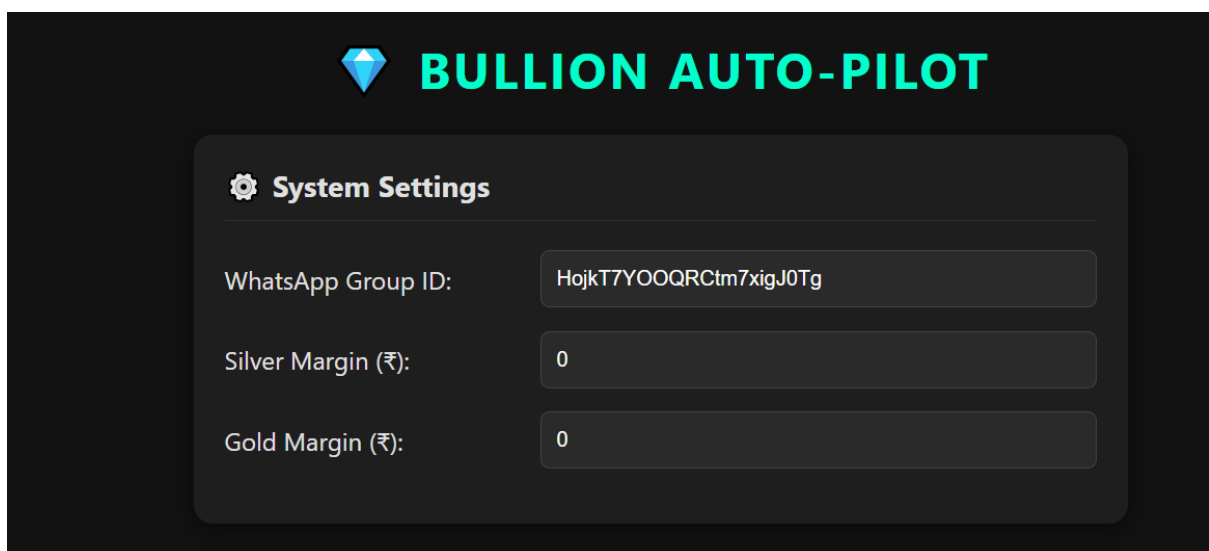


The screenshot shows a 'Daily Schedule' interface. It features three input fields for update times: 'Morning Update' set to 02:50 PM, 'Afternoon Update' set to 03:00 PM, and 'Evening Update' set to 03:05 PM. Each field has a clock icon for editing. Below these fields are two large buttons: a green '▶ START BOT' button and a red '■ STOP BOT' button.

Daily Schedule	
Morning Update:	02:50 PM
Afternoon Update:	03:00 PM
Evening Update:	03:05 PM

▶ START BOT ■ STOP BOT

2.System setting



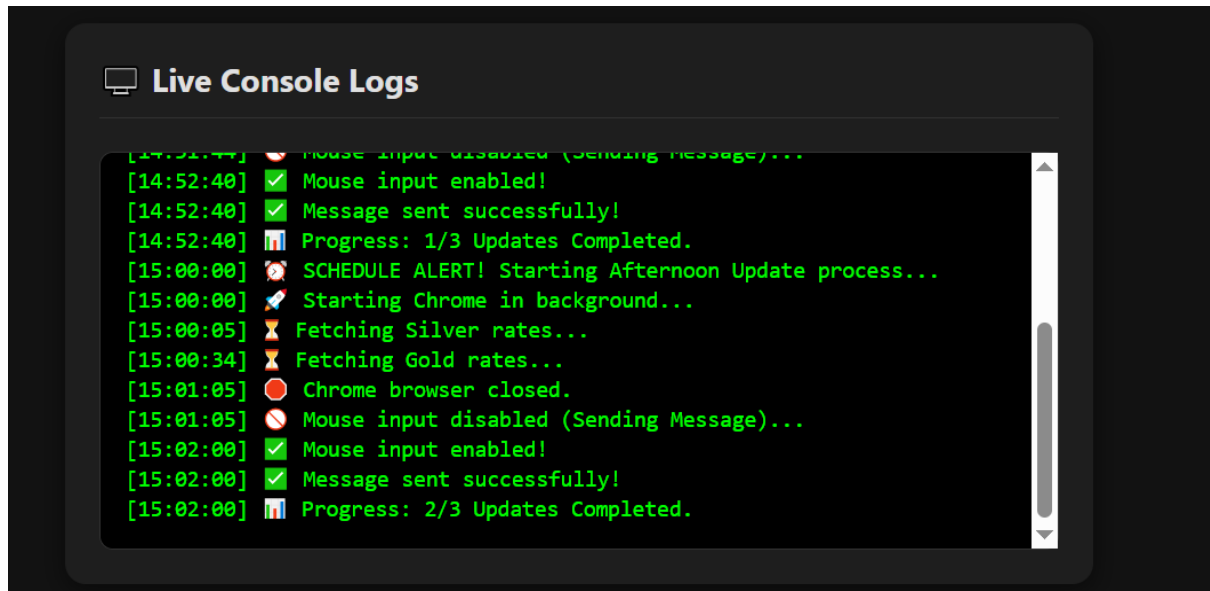
The screenshot displays the 'BULLION AUTO-PILOT' system settings. The title is in large green letters with a diamond icon. Below it, the 'System Settings' section contains three input fields: 'WhatsApp Group ID' with the value 'HojkT7YOOQRCtm7xigJ0Tg', 'Silver Margin (₹)' with the value '0', and 'Gold Margin (₹)' with the value '0'.

BULLION AUTO-PILOT

System Settings

WhatsApp Group ID:	HojkT7YOOQRCtm7xigJ0Tg
Silver Margin (₹):	0
Gold Margin (₹):	0

3.Live console



10. Limitations

Every system has certain limitations. The limitations of the Bullion Auto-Pilot System are as follows:

- The system depends on internet connectivity for fetching live rates and sending messages.
- Web scraping may fail if the structure of the source websites changes.
- The system uses WhatsApp Web automation, which may require manual login or may stop if the session expires.
- The system does not store data permanently as it uses in-memory processing.
- The application does not include user authentication or security features.
- The system is dependent on third-party libraries such as Selenium and PyWhatKit, which may have compatibility issues in future updates.

11. Conclusion

The Bullion Auto-Pilot System successfully automates the process of fetching and sharing gold and silver rates. The system reduces manual effort by automatically collecting live data from websites, applying user-defined margins, and sending updates to WhatsApp groups at scheduled times.

The project demonstrates the effective use of technologies such as Python, Selenium, and Flask for real-time automation. It improves accuracy, saves time, and ensures timely communication with customers.

Although the system has some limitations, it provides a strong foundation for future enhancements such as database integration, user authentication, and API-based data fetching.

Overall, the system proves to be efficient, reliable, and useful for bullion shop owners.

12. Bibliography

- Python Official Documentation – <https://www.python.org>
- Flask Documentation – <https://flask.palletsprojects.com>
- Selenium Documentation – <https://www.selenium.dev>
- PyWhatKit Documentation – <https://pypi.org/project/pywhatkit>
- W3Schools – <https://www.w3schools.com>
- Stack Overflow – <https://stackoverflow.com>